

ON BUILDING

SYSTEMS

Production LLM Systems



#ASLI ENGINEERING

BY

ARPIT BHAYANI

Agenda

1. Graceful Degradation
2. Prompt Caching
3. Observability and Cost Attribution
4. prod gotchas

Prototypes: 2

System : Natural language Workflow Engine

Appendix:

- Architecture Decision Framework
- Streaming

Graceful Degradation and Fallback

Rate limit and provider outages are not **if** but **when**!

regular occurrence... graceful handling or collapse!

1. priority ordered list of models - Opus → Sonnet → GPT-4o → ...

* depending on task it could even be: Sonnet → opus → ...

* different models yield diff outputs preference to cost

↓ decide per feature whether degraded response or no response

ex: chat → shorter reply is better than no reply

math puzzle solving degradation is not okay

2. Circuit Breaker if too many errors, it may be okay to stop everything and show a 'cute' unavailable message

✓ v/s Sxx errors
or pause pipeline

3. No unlimited retries (retry storms are reality, be safe)

↳ jitter

↳ Respect sleep = Rate limit response headers

→ caching input, not output (LLM response)

Prompt Caching

→ for some usecases

Highest leverage optimization → easy savings

prompt → provider computes
prefix hash

value vector internal state attention layers

if same prefix seen recently, KV cache from earlier request used.
* skips attention computation over cached token!

50,000 token system prompt

100 req/hour

without caching $50,000 \times 100 = 5,000,000$ tokens/hour

10% cache miss
TTL, eviction, etc

with caching (assuming warm), we pay $50,000 \times 10 \approx 500,000$ tokens

TTL ← some misses and non-sys prompt
tokens will be there for inference.

what gets cached? → leading portion upto declared cache breakpoint

Prototype: Prompt Caching

Model	Cache Read (per 1M tokens)	Storage Price (per 1M tokens/hour)
Gemini 3.5 Flash	\$0.15	\$1.00
Gemini 3.1 Pro Preview	\$0.36 (≤200K tokens) / \$0.72 (>200K tokens)	\$8.10
Gemini 3.1 Flash-Lite	\$0.045 (text/image) / \$0.09 (audio)	\$1.80
Gemini 2.5 Pro	\$0.125 (≤200K tokens) / \$0.25 (>200K tokens)	\$4.50

Open Telemetry for AI

latency and status code

Traditional APM treats LLMs as blackbox HTTP endpoints

OpenTelemetry GenAI Semantics bridges this gap

↳ standardizes tool calls, llm calls, and retrieval steps

Each LLM call is a span with standardized attributes

gen-ai.system: provider

gen-ai.request.model

gen-ai.usage.input-tokens

gen-ai.usage.output-tokens

gen-ai.response.finish-reason

* when we use a compliant library
dashboards look identical weather
call went to anthropic, openAI, google

Tool gets its own child span → tool name and input

- how long did model take to decide to call this tool?
- how long did tool take to execute?
- which tools are called most frequently

Multi-step execution

root span → user request

child span → each agent iteration

grand child → each llm call

great-grand child → tool calls

Exactly what agent did
which step was slowest
which step cost the most

Cost Attribution  avoid surprises!
helps you decide pricing/billing

Tag every LLM call on at least 3 dim: user-id, feature-name, step

- * which 5 users are responsible for 40% of your spend?
- * which features consumption grew 3x last week?
- * which step is burning most token?

Use OpenTelemetry compatible SDK

attach custom span attributes before each call

export to observability backend - say Langfuse (ClickHouse)

Alerting: set alerts at three levels

1. per-user spend rolling over 24 hour window
2. per-feature spend rate (catches prompt regression)
3. per-agent step spend rate

Fire alerts to the team who owns it.

 cut-off if breach

Budget Reinforcement: send 429 even before LLM call is made

Prototype: Observability

Natural Language Workflow Engine

accepts plain text description of desired automation

decomposes into ordered steps

selects appropriate tools for each step

runs workflow in sandbox with checkpoint and resume

Design Pressure: Co-ordination Heavy + Correctness

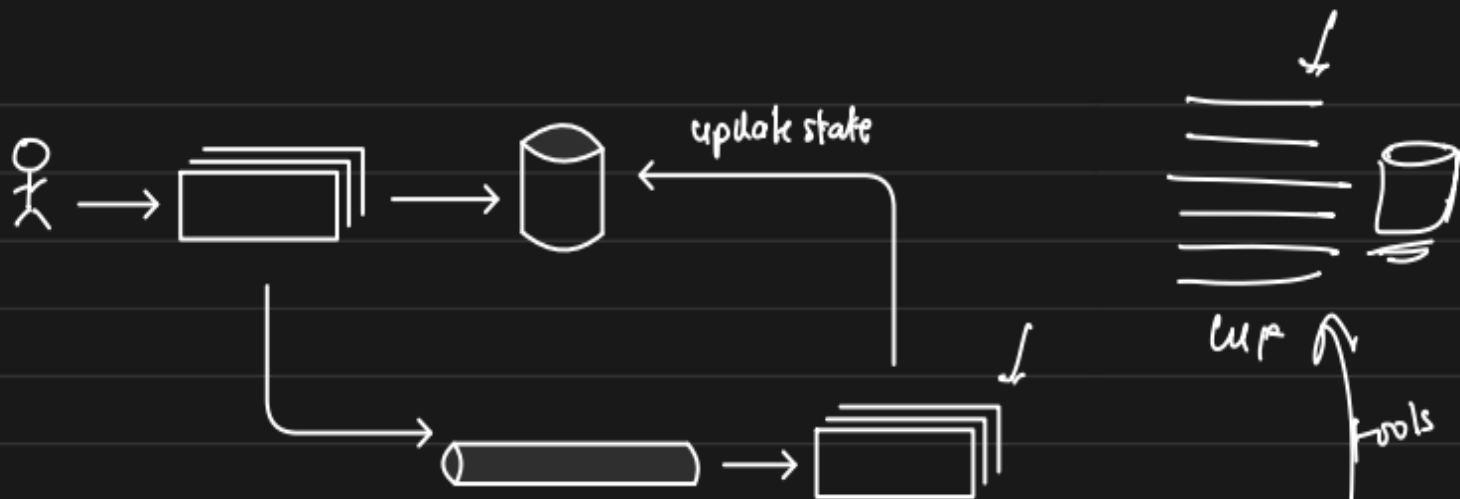
Solutioning...

1. user journey, workflow state management
2. API server responsibility
3. How to transform workflow into steps? correctness >>> completeness >>>
4. Editing a WF $\rightarrow$ what are steps?
5. Connectors

Every day, get top AI news from HN, and send me an email

6. Workflow execution loop
 - $\rightarrow$ shared conv history v/s isolation for each step

"Test mode"



Feasibility = Tool stw

Each step: input/output

Step types } limited
↳ "standards"

Schedule | Action Trigger

→ agent call

↳ function calling tool registry
choice ~ branching
loop step - -

WF
↳ steps
msg = [. - .]

$$F_k$$

гмд. мсп

guard rail $\rightarrow$ check if task is
 step ① from my domain

WF	Step		A
id	id	wf_execut	step_exec
inst	wf_id	id	id
	instruc	wf	wf_exe
	tools		skp_id
			stak

ARPIT BHAYANI



1. create, delete, get wf
2. transform workflow into steps
3. edit a workflow

workflow

id
instructions
state : DRAFT
⋮

DRAFT

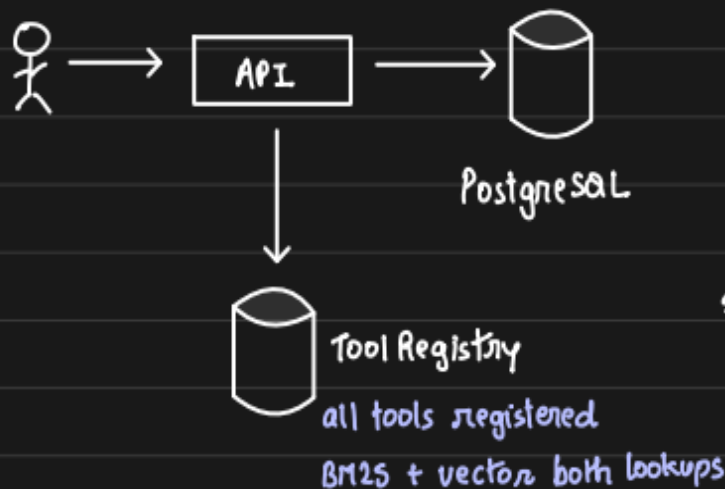
READY-TO-TEST

READY-TO-RUN

PAUSED

DELETED

workflow to steps



instructions + tools available



RAIL on all

1. smart model to breakdown
2. iterate multiple times
3. fetch relevant tools for each step
4. each step should be self-sufficient
5. LLM call to verify
 ↳ steps do solve the workflow
 ↳ multiple LLM calls
6. mark WF - ready to test state

Editing a workflow

Edit natural language instruction
and regenerate the wf steps.

our requirement → bare min^m change to existing wf

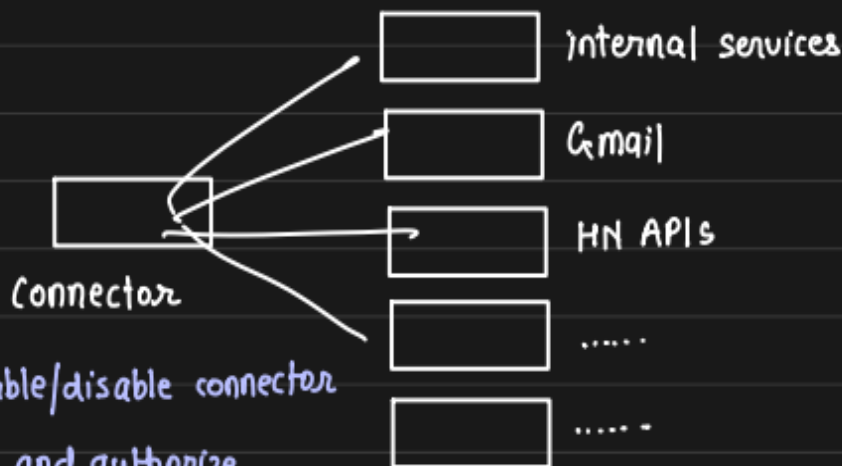
1. current wf steps
 2. new instructions
- generate new steps

double check with LLM

Connector

"Every day, get top AI news from HN, and send me an email"

- ↳ step1: setup CRON if it does not exist
- ↳ step2: make call to HN and get all top 100 stories in a day
- ↳ step3: go through each story and see if AI related and 'top'
- ↳ step4: compile summary
- ↳ step5: send email



composio-dev

nango-dev

cx to enable/disable connector
configure and authorize

- API key and OAuth

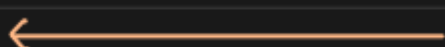
* onboards every operation to itself via MCP

↳ connector DB becomes tool registry

* exposes /invoke

↳ invokes tool/operation as suggested by LLM

Workflow Execution loop

messages = []  shared conversation across all steps

For each step in workflow: on each step is its own independent
 step.execute() execution with its own context

Trade-off: large number of steps → context window overwhelm
no need for data passing semantics across steps

Approach 2: Each step run in isolation w/ its own context window

↳ each step should have its output message well defined

↳ O/p of all prev step go to current step as context

↳ no need to be selective here

def orchestrator():

 messages = []

 for step in workflow:

 msgs = step.execute(messages) →

 messages.extend(msgs)

 db.update(step, COMPLETE, messages)

Each step is multi-step
with tool calls and synthesis
depending on instructions
of that step

Transitive Errors, Root causes,
and potential fixes

Production Gotchas

Long Running Conversation / Loop : Brainstorm

Side-effect → context window blocks up

- cost, latency
- lost in the middle
- because of eviction, you are losing ctx

- instruction drift

- context poisoning

→ why this could have happened? Transitive WHYs...

- multi-step
- all in one loop
- oscillation
- lack of limits / lack of success criteria
- retry loops / tool request behavior

goal drift

Tools call result, method,

Seq tool call / # tool calls

Cx is unhappy / dissatisfied

Cx is chatty

→ Mitigation

subagent (isolated context)

max turns, time, token, cost (at step / overall)

"checkpoint"

to prevent poisoning

Summarize event, eliminate tool call result

sliding window

lossy →

Long Running Conversation/loop :

- context window blocks up
- cost (entire history sent every time)
- latency
- hit context limit
- lost in the middle
- context poisoning
[bad tool call, response
stays in the conversation]
- same pattern repeated
- instruction drift

→ why this could have happened? Transitive WHYs...

- multistep tasks
- all baked in one agentic loop
- ambiguous success criteria
- retry loop
- goal drift (because of some subtask)
- oscillation (between A & B)
- lack of max steps / budget
- Tools too fine granular
- user is chatty.

→ Mitigations:

Periodic summarization → but lossy → context dilution

Eviction, external mem,

sub-agents (with, separate context)

trimming tool outputs (because already consumed)

Brainstorm

Things that could go wrong with a Negotiation Agent
and ways to fix that....

discount refund
0 - 15% 15%.

- sycophancy attack
 - ↳ emotional / blackmail...
- goal / inst drift
- commit - 20% false promise
- extract - strategy / proxy / tier
- stock weak
- reverse engineers rude / not empathetic

Way fix:

guardrail
say ults commit

sanity
max turns in negotiation

walk away criteria
phone call

Things that could go wrong with a Negotiation Agent and ways to fix that....

1. opens too weak (offers more discount to begin with)
 2. cx can reverse engineer strategy & can manipulate
 3. commits to terms outside of scope
 4. Sympathy
 5. comes as rude / pushover
 6. leaks strategy / prices / tiers
- ↳ blackmail
emotional situ

ways to fix:

- Have a walkaway point
- max turns for negotiation (voice is not free as well)
- sanity check (violate hard constraint)
- Separak - what to say v/s what to commit

Downstream impact of retry loop and a swarm of subagents and how to deal with it...

Brainstorm



retries $\rightarrow$ bloot up telemetry $\rightarrow$ cost
experience degrade $\rightarrow$ infra

connection exhaust
- agent specific conn pool

agent v/s user traffic gets affected

How to fix $\downarrow$
read only load
then replicas for agents

"same tool calls with
diff param"

"Isolation"

How to fix:

cap your retries

Sep conn pool
Sep read replicas (cache (stale))

agent specific role RBAC
 $\hookrightarrow$ circuit break

Downstream impact of retry loop and a swarm of subagents and how to deal with it...



connection pool exhaustion
lock contention due to retries

Alert storms

How to deal:

- cap on max retries
- global concurrency limit ($\# \text{workers} / \text{agents}$)
- circuit breaker
 - ↳ SQL commenting
- separate agent roles in DB

Brainstorm

Meeting Scheduling agent, too much of back and forth, ...

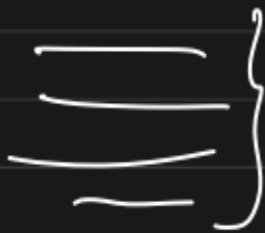
what could go wrong?

how to fix it?

- expose user meeting
 - long conversation → agent confuses
 - staleness → infinite ping-pong
- busy/not busy with X
- guardrails
- working hours
 - 'never double book'
- spams

How to fix it

- hard guardrails 'day, timezone,'
 - agent cannot "read" meeting
 - max iter/cutoff ... exit criterion
 - summarize
- state limit



- multiple slots @ once

Meeting Scheduling agent, too much of back and forth, ...

what could go wrong?

how to fix it?

- infinite ping pong
- stale state (because other meeting scheduled)
- double booking in same slot
- information leakage (other meeting scheduled with X)
- No exit condition

How to fix it?

- ask once for hard constraints ^{↗ day range, time, timezone, etc}
- propose multiple options per iteration
- Busy v/s Meeting with X $\longrightarrow$ agent should not know meeting is with whom
- keep summarizing previous conversations

So far we discuss...

— —
— —
— —

none of these work

} Send these in
each response

Appendix

prototype v/s production - 1000 commits

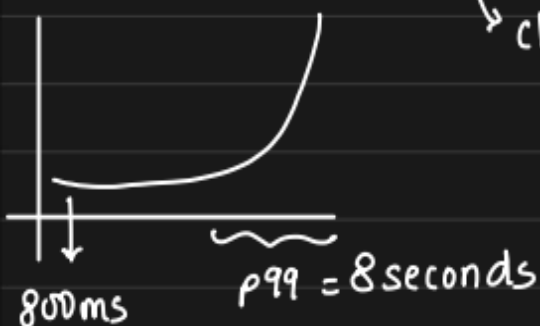
Architectural Decision Framework

understand product latency requirements } undoing a bad arch decision
workload profile } is expensive!

Synchronous calls: client blocks $\rightarrow$ model responds $\rightarrow$ response returned

↳ chat interface, single-turn QnA, etc

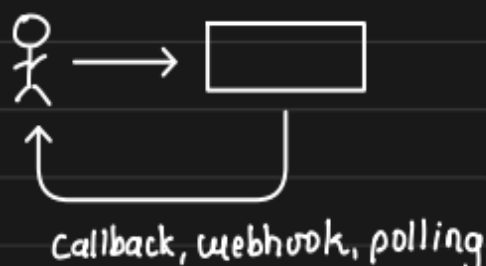
(docs website QnA bot)



we are exposing tail latencies to end users.

Asynchronous Invocation:

great for non-user facing pipelines and processing.



We can retry!
without any
cascading failures.

ex: document summarization,
nightly classification runs,...

Batch mode: cost optimized async (OpenAI/anthropic batch API)

↳ 10s - 1000s of prompts in one go, hours later response 50% ↓ cost

ex: eval runs, data labelling, embedding generation on corpus

Streaming

Yes, it makes ux awesome
but it is an important arch decision

→ implications on cost, error handling, and infrastructure.

Tokens are streamed over HTTP using SSE (server sent events)

↳ emitted as and when tokens are generated 800ms → 8sec

1. connection drop midway (half sentence, half json object)

↓
clients must handle this gracefully

display what arrived, show error indicator, offer a retry

2x cost

↳ sends full prompt again

2. Cancellation (navigates away / submits new message before stream ended)

↳ we should cancel inflight request (abort controller)

saves cost + better ux (not making user wait)

3. while streaming structured data

↳ retry again

4. Back pressure

↳ system and model can handle 200 concurrent streaming

but we got 1000 requests

enqueue them and ack waiting (no error no abort)